Autaptic inhibition's role in continual learning

Jens Friedel
School of Engineering Mathematics and Technology
University of Bristol
Bristol, United Kingdom
jens.friedel@bristol.ac.uk

Conor Houghton
School of Engineering Mathematics and Technology
University of Bristol
Bristol, United Kingdom
conor.houghton@bristol.ac.uk

Gabriel Leivas Oliveira
School of Engineering Mathematics and Technology
University of Bristol
Bristol, United Kingdom
gabriel.leivasoliveira@bristol.ac.uk

***Abstract*—How can one design a performant, memory-efficient continual learning framework that operates within a network's fixed capacity? We hypothesize that local capacity constraints limit long-term learning performance in sparse networks, and how distributing learning schemes could circumvent this problem. Inspired by biological networks, we propose *autaptic inhibition for artificial intelligence (ai2)* as a mechanism to motivate networks to distribute representations across many neurons. We then lie the theoretical and implementation groundwork to test whether they can outperform their conventionally sparse counterparts by balancing knowledge retaining and information redistribution flexibly.**

***Index Terms*—Continual learning, Catastrophic forgetting, Neural network architecture, Biologically inspired systems**

I. Introduction

Continual-learning systems, which are to acquire new representations over time, face *catastrophic forgetting*, where adapting parameters to new data degrades previously learned representations [1]. One framing of memory-efficient continual learning's research question is thus: How can a network of fixed size keep learning without sacrificing past performance?

We hypothesize that a major contributor to forgetting is local capacity exhaustion. When a single neuron is responsible for differentiating a feature, it can't be used to store novel representations. At the same time, neurons responsible for detecting features of a particular domain tend to live in similar network regions. We argue the exhaustion of that neighbourhood's representation capacity suffices to limit network performance irrespective of the total network capacity.At the same time, similar features . Conversely, sidestepping local capacity exhaustion would unlock more network capacity for representation learning, enabling better continual learning performance.

Is there a ref for this?

In contrast to sparse networks, distributing a representation across many neurons ensures the local network part responsible for a feature domain covers a larger part of the network and, subsequently, has access to more of the network's total capacity.

To encourage this distribution, we take inspiration from **autapses** — self-recurrent synapses observed in biological neurons that let a neuron inhibit its own future activity based on its recent firing. We propose **autaptic inhibition for artificial intelligence** (ai2): a self-recurrent inhibitory feedback term on a layer's neurons, driven by each neuron's own recent output, intended to discourage any one neuron from acting as a stable, unshared feature detector. To our knowledge, there is no prior work applying self-recurrent, autaptic connections to distribute representation learning in continual learning settings.

Because we actively push the network from its usual learning trajectory, we expect worse accuracy at the beginning. Yet, we expect the lack of sparsity to enable slower and more graceful forgetting, since no single neuron's reuse can erase a whole feature. We assume a well-calibrated ai2 implementation balances initial performance loss and a more flexible network storage to improve overall continual learning performance.

II. Related Work

We use the Avalanche framework [2] as a general reference for continual learning, in particular regarding benchmarks and evaluation protocols; though it currently is not a project dependendy.

Meta-learning and continual learning are increasingly combined to let a network's own learning rule adapt over a task stream [1]; our approach is complementary and structural rather than rule-based — we change what a neuron's output **can** represent, by coupling it to its own recent activity, rather than changing how parameters are updated.

Related Work target: 0 column, cap hard – do not let this grow past a short paragraph. Remaining: a biological-autapse citation, and (if found) the RNN self-recurrence citation flagged above. If no time, cut to 3 sentences and merge the biology citation into the Intro autapse mention instead.

III. Method

A. Autaptic Inhibition Mechanism

An `AutapticLayer` replaces a plain linear-plus-activation hidden layer. In order to allow the autaptic inhibition to depend on the current input during learning, each forward pass unrolls the same, fixed input over T internal pseudo-time steps. During each pseudo-time step, a neuron's autapse feeds back the last neural output, which the neuron uses to weaken its own output.

For now, we implement two **variants** combining this feedback with the neuron's original activation, as well as two **schemes** dictating how to integrate the inhibition signal with the to-be-inhibited output.

We write x for a neuron's (unroll-step-invariant) external input and a for the layer's activation function. We assign $y(t)$ to a neuron's uninhibited activation at step t, $y_{\text{self}}(t)$ to its autaptic inhibition signal, and $\tilde{y}(t)$ to its ai2-adapted output.

The former two are only used within the neuron:

$y_{\text{self}} \leftarrow 0$
$\tilde{y} \leftarrow a(z)$
for $t = 1$ **to** T **do**
  $y_{\text{self}} \leftarrow g(\tilde{y})$
  $y \leftarrow a(z)$
  if variant is ai2_diff **then** $\tilde{y} \leftarrow y - y_{\text{self}}$
  else $\tilde{y} \leftarrow y(1 - y_{\text{self}})$
  end if
end for
return $\tilde{y}$

Algorithm 1. Forward pass of one `AutapticLayer` (per neuron; every neuron's autapse acts independently). Degenerates to a plain activated linear layer at $T = 1$, since $y_{\text{self}}(0) = 0$.

$$\begin{aligned} y(t) &= a(x) \\ y_{\text{self}}(t) &= g(u), \quad y_{\text{self}}(0) = 0 \end{aligned} \tag{1}$$

Under the `sigmoid` scheme, a is a sigmoid function and $g(u) = u$; under `relu_sigmoid`, a is ReLU and $g(u) = \tanh(\frac{u}{2})$, which keeps the feedback signal bounded to $[0, 1)$.

The two variants then differ in how $y(t)$ and $y_{\text{self}}(t)$ combine: **ai2_diff** subtracts the feedback,

$$\tilde{y}(t) = y(t) - y_{\text{self}}(t), \tag{2}$$

while **ai2_gate** multiplicatively gates it,

$$\tilde{y}(t) = y(t)(1 - y_{\text{self}}(t)). \tag{3}$$

A neuron finally outputs $\tilde{y}(T - 1)$ (Algorithm 1) to the next layer.

Since $y_{\text{self}}(0) = 0$, both variants leave $\tilde{y}(0) = y(0)$ unchanged. Consequently, $T = 1$ degenerates **exactly** to a plain activated linear layer.

B. Continual-Learning Protocol

We adopt a class-incremental protocol on MNIST: each run starts with digits $\{0, ..., 4\}$, then introduces one of the remaining five digits per stage in a total of six stages. A stage's training set is the union of all digits introduced so far, so that the network must genuinely retain earlier digits while learning new ones.

A stage's duration is fixed to a number of epochs instead of a loss threshold, since a loss-based stopping criterion could over-constrain which regions of the loss landscape a run can explore.

Every stage is evaluated on the full test set: Already-introduced digits give a forgetting read-out, not-yet-introduced digits a generalization read-out.

C. Metrics

In every stage, we track: the per-digit test-accuracy matrix; average accuracy over introduced digits; backward transfer, the mean accuracy drop between stage 0 and the final stage on stage 0's digits (the standard forgetting measure); and the mean absolute value of each `AutapticLayer`'s feedback signal y_{self} over an epoch, as a first, coarse proxy for how distributed a layer's activity is. We do not yet have evidence that this proxy tracks distributedness correctly (Section V).

D. Implementation and Experimental Setup

The pipeline separates data staging, model definitions, the training loop, MLflow-based logging, and sweep/run orchestration into independent modules.

We compare three controls (a CNN; an FCNN per activation scheme) against the 2×2 ai2 variant/scheme grid at a fixed T, plus a one-factor-at-a-time sweep over T, epochs per stage, and hidden-layer width to keep the search tractable.

Next to the metrics mentioned in Section III.C, a run logs the per-epoch loss, per-stage accuracy, and y_{self} absolute means to a local MLflow instance.

IV. Preliminary Observations

V. Discussion and Limitations

Everything reported so far is implementation and design, not yet a validated result, for the following reasons.

First, every run so far used a single seed. Data-loader shuffling draws from one global RNG stream that different configurations consume at different rates, so even same-seed runs are only guaranteed to match through weight initialization and the first training epoch. Seed-replicated runs, already supported via the logging module's replicate-group mechanism, have not yet been collected.

Second, the y_{self} absolute-mean metric is a heuristic to see if autaptic inhibition takes place. After wrapping up a systematic exploration of system behaviour, we are going to record every neuron's autaptic inhibition to see how local dynamics evolve over time.

Third, several design decisions focused on getting results quickly and are going to be superseded in the coming weeks: the anchor configuration for the one-factor-at-a-time sweep, the sweep's value grids, and seeding each run with five (rather than, say, three) initial digits; leaky memory of past activity shall supersede T pseudo-time step unrollings; the ai2 variants and schemes are neither final nor well-calibrated.

Also note that an early comparison using Adam produced practically indistinguishable control and ai2 runs, which we attribute to Adam's adaptive learning rate absorbing the comparatively small change ai2 makes to the loss landscape. As a response, the optimiser switched to plain SGD, in order to have the learning be more sensitive to the local loss shape.

Lastly, past runs used class-incremental stages which revealed the majority of feature space in the first stage. The control networks may not learn new representations, but only need to change weights, meaning the runs hardly evaluate continual representation learning.

VI. Conclusion and Future Work

We have implemented and tested a self-recurrent inhibitory mechanism (ai2) intended to push networks toward distributed, rather than locally concentrated, representations, and

Target: 0.6-column. This is the one section that depends on today's work, not just on writing – pick ONE of the two paths below and commit early, since it gates the Discussion section tone:

- **If**
- **If**

built a class-incremental MNIST pipeline to evaluate it. The pipeline runs end-to-end and is unit- and integration-tested; what remains is a through analysis to obtain evidence that ai2 improves continual-learning outcomes over its controls.

The immediate next steps are seed-replicated runs for variance estimates; sweeps over more constrained network and training budgets, where an ai2 benefit, if any, should be clearest; a learning-rate sweep; and a genuine activity-recording metric alongside y_{self} to analyse local dynamics.

References

[1] J. Son, S. Lee, and G. Kim, "When Meta-Learning Meets Online and Continual Learning: A Survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024, doi: 10.1109/TPAMI.2024.3463709.

[2] A. Carta, L. Pellegrini, A. Cossu, H. Hemati, and V. Lomonaco, "Avalanche: A PyTorch Library for Deep Continual Learning," *Journal of Machine Learning Research*, vol. 24, no. 363, pp. 1–6, 2023, [Online]. Available: http://jmlr.org/papers/v24/23-0130.html